

028 实验：检查一个广播自动微分器

目标与文件

本实验把026的标量图扩展成小型NumPy张量图，并明确它的边界。你要验证广播转置归约、共享节点、显式向量种子，随后用不可微点、梯度屏障和二阶导数反例说明自动微分没有保证什么。

先修027和014。engine.py实现Tensor、sum_to_shape与vjp；experiment.py运行固定合成诊断并保存五份输出；test_experiment.py使用与引擎不同的索引、分数、高精度和差分参照。没有训练或测试集，也没有真实分类任务。

在本单元目录运行：

```
conda env create -f environment.yml
conda activate dl-unit-028
python experiment.py --output outputs
python test_experiment.py
jupyter lab
```

Notebook重启内核后从第一格运行，写notebook_outputs并与随附五文件逐字节比较。实际验证Python3.12.14、NumPy2.3.5、新进程和真实InProcessKernel；全新Anaconda、浏览器Jupyter、外部socket传输、GPU与跨平台安装未测试。JAX、PyTorch只读官方文档，不是本单元依赖。

实验A 从一个完整雅可比认识seed

纸笔计算 $F(x_1, x_2) = (x_1 x_2, x_1 + x_2^2)$ 。在(2,-1)处先写输出(-2,3)，再逐项求 2×2 雅可比。行必须代表输出、列代表输入。给seed=(3,-1)和方向t=(1/2,2)，分别求 $J^T v$ 与Jt，最后检查两种内积均为14。

下面用两个常数掩码组成一个向量输出，不需要额外实现可微拼接算子：

```
import numpy as np
from engine import Tensor, vjp
x1, x2 = Tensor(2.), Tensor(-1.)
out = x1*x2*np.array([1., 0.])
out = out + (x1+x2*x2)*np.array([0., 1.])
g = vjp(out, [3., -1.])
print(out.value, g[x1], g[x2])
assert float(g[x1]) == -4.
assert float(g[x2]) == 8.
```

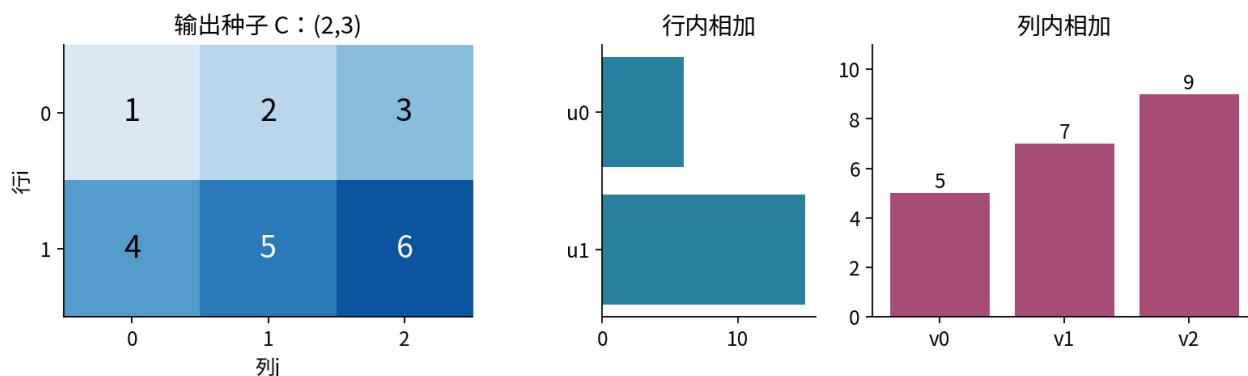
再把seed改成[1,1]，解释它对应哪个标量目标。调用vjp(out)应报错；向量输出没有唯一自动选择的梯度。一个元素的向量shape(1,)也与标量shape()不同。

实验B 把广播的反向写成索引求和

设 $u=[[1],[2]]$, $v=[[0.5,-1,1.5]]$, $Z=u+v$ 。给种子 $C=[[1,2,3],[4,5,6]]$ 。不要先调用`sum_to_shape`, 先用双重循环把每个 $C[i,j]$ 加回来源 $u[i,0]$ 与 $v[0,j]$ 。

纸笔预期为 u 梯度 $[[6],[15]]$, v 梯度 $[[5,7,9]]$ 。然后运行：

```
from engine import Tensor, vjp
u = Tensor([[1.], [2.]])
v = Tensor([[.5, -1., 1.5]])
g = vjp(u+v, [[1., 2., 3.], [4., 5., 6.]])
print(g[u], g[u].shape)
print(g[v], g[v].shape)
assert g[u].shape == (2, 1)
assert g[v].shape == (1, 3)
```



图A 按来源相加。梯度值和原始shape都属于验收条件。

再把 $u+v$ 改成 $u*v$, 先乘各位置保存的另一个输入, 再沿复制轴求和。推广到 $A(2,1,3)$ 、 $B(1,4,1)$, 写明 A 沿`axis1`, B 沿`axis0`和`2`归约。若结果“还能广播运行”但`shape`不对, 仍然判错。

实验C 检查主图的每个坐标

默认 $X(2,3)$ 、 $w(3,)$ 、 $b()$ 来自`data/config.json`。模型 $A=X*w+b$, $H=\tanh(A)$, $Z=H*H+H$, $L=\text{sum}(Z)/6$ 。 L 是微分检查目标, 可能为负, 不能解释为真实预测误差。

1. 手算 A 的六个元素; 应包含 $-3/8$ 、 0 、 $7/8$ 、 $1/8$ 、 $-1/8$ 、 $-1/4$
2. 对均值目标推导 $Q=(2H+1)(1-H^2)/6$, 核对 w 与 b 的归约
3. 单独核对 $A[0,1]=0$ 处, 对 $X[0,1]$ 的梯度为 $-1/24$
4. 打开`coordinates.csv`, 将 $X/w/b/a/h/z/L$ 的数值和梯度`shape`对上
5. 用给定非均匀`seed`对 Z 调用`vjp`, 解释 b 梯度从约 0.81839 变为 -0.20840 的原因
6. 保留`finite_difference.csv`全部90行, 而不是只挑误差最小的一行

完整测试另外使用360个精确分数广播图、40个完整数值雅可比及内积检查、121个高精度激活点和49个多项式点。检查这些参照是否真的走了不同计算路线。

实验D 三个故意不“对齐”的结果

D1 不可微节点组合

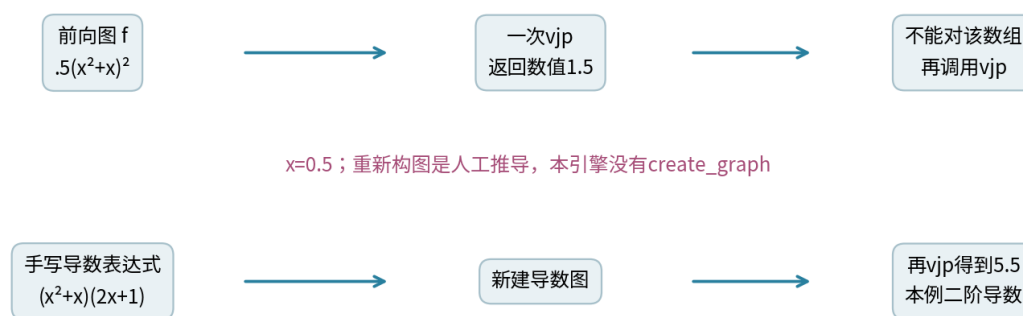
令 $x=0$ ，构建 $\text{ReLU}(x)-\text{ReLU}(-x)$ 。数值函数恒等于 x ，普通导数为1；本引擎按两个ReLU零点局部约定返回0。写出两条局部路径各自的值，不用“有误差”搪塞这一差异。

D2 显式梯度屏障

令 $x=2$ ，比较 $x*x*x$ 与 $(x*x).\text{detach()}*x$ 。二者前向均8，返回导数分别12与4。说明4来自主动停止某条依赖，而非普通 x^3 的导数。再把源NumPy数组创建Tensor后改掉，确认已建图不被改变。

D3 高阶导数需要导数图

令 $x=1/2$ ，目标 $f=(x^2+x)^2/2$ 。一阶应为1.5。直接对`vjp`返回的NumPy数组再求导应拒绝；请手工写出 $df=(x*x+x)*(2*x+1)$ ，对这张新图求一阶，得到5.5。明确哪一步是你自己推导并重建的，不能声称引擎已经支持自动高阶微分。



图B 两条路线区别在于是否真的存在导数计算图。

实验E 可复现失败与提交

将配置复制到`custom.json`，改动一个合法数值，使用`--config custom.json --output custom_outputs`另存。固定Notebook和图在默认文件变化时会拒绝运行，这是为了防止文字与图不再匹配。

在临时配置里分别放入`bool`、`NaN`、`1e-400`、错形状、重复键或额外键，检查错误发生在写结果之前。已有输出五文件应保持原样；尚不存在的新目录不应被创建。晚期序列化失败、输出符号链接也有自动测试。这个契约不包括操作系统写到一半时的多文件原子回滚。

交付内容：完整手算雅可比、广播索引账本、顺序执行Notebook、五份匹配结果、三个边界案例解释，以及“梯度正确仍没有证明什么”的短文。至少区分建模目标、优化、数据拆分和真实表现四种责任。详解在`answers.pdf`。